Science of Computer Programming 236 (2024) 103115

Contents lists available at ScienceDirect

Science of Computer Programming

journal homepage: www.elsevier.com/locate/scico

Agile effort estimation in Colombia: An assessment and opportunities for improvement

Juan Cubillos *, Jairo Aponte, Diana Gomez, Edwar Rojas

Facultad de Ingeniería, Universidad Nacional de Colombia, Ciudad Universitaria, Bogotá, Colombia

ARTICLE INFO

Keywords:
Effort estimation
Agile effort estimation
Agile software development
Software development effort estimation
Software measurement

ABSTRACT

Effort estimation is fundamental for the development of software projects and critical to their success. The objective of this paper is to understand how Colombian agile practitioners perform effort estimates and to identify opportunities for improvement based on these results. For this purpose, we conducted an exploratory survey study using as instrument an on-line questionnaire answered by agile practitioners with experience in effort estimation. Data was collected from 60 agile practitioners and the main findings are: (1) Agile practitioners prefer non-algorithmic estimation techniques, mainly those based on Expert Judgment. (2) Most of the respondents perceive that their estimates have a medium accuracy level; however, in most cases, no formal analysis of the accuracy level is carried out. (3) The determining effort predictors/cost drivers are characteristics of the project team (size, experience, and skills) and attributes of the software to be built (complexity, type, and domain). (4) The use of datasets for estimation is not common; proprietary datasets predominate and are used for productivity comparisons within the company. (5) Most of the results of related studies are comparable with ours; however, there are significant differences in terms of the roles involved and the techniques used in the effort estimation process. Based on the results and findings of the survey, we identified key opportunities to improve estimation accuracy through (1) software measurement standardization, (2) use of effort datasets, (3) implementation of techniques for measuring accuracy levels, and (4) knowledge management in effort estimation.

1. Introduction

Software Development Effort Estimation (SDEE) is the process of assessing or predicting the most realistic effort, duration, and cost of a software endeavor [2], [3]. In the software engineering domain, effort estimation and planning are two closely related activities since the first one is essential to carry out the second one [4]. Accurate and precise estimates influence several planning tasks that help complete the project properly, such as creating a detailed schedule, breaking down a project into phases, and prioritizing deliverables.

Although effort estimation has been widely studied, the iterative nature of agile methods requires specialized estimation techniques [5]. Despite the numerous approaches proposed so far, the effort estimates in Agile Software Development (ASD) are far from perfect, and in most cases the tendency is to underestimate the effort required. Requirements, management, and team-related issues are cited as the top causes for large differences between the actual and estimated effort [6].

* Corresponding author.
E-mail addresses: jmcubillosa@unal.edu.co (J. Cubillos), jhaponte@unal.edu.co (J. Aponte), dgomez@unal.edu.co (D. Gomez), erojas@unal.edu.co (E. Rojas).

https://doi.org/10.1016/j.scico.2024.103115
Received 15 September 2023; Received in revised form 28 March 2024; Accepted 1 April 2024

Available online 5 April 2024
0167-6423/© 2024 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/).

Expert Judgment techniques have been by far the most common estimation approach used in Agile Effort Estimation (AEE) to predict software development effort. Expert Judgment is the assessment based on the experience in an application area, knowledge field, discipline, or industry, for the activity being performed. Such expertise may be provided by groups or individuals with specific education, knowledge, skills, experience, or training [7]. Planning Poker, the most widely used estimation technique in ASD, is a variant of the Delphi method. This method itself is a specialized form within the broader category of expert judgment techniques.

A recent update of a Systematic Literature Review (SLR) on effort estimation in ASD summarizes the results of primary studies published from December 2013 to January 2020. The review focuses on 73 articles that provided new evidence to characterize estimation activities in agile projects. These results confirm that Planning Poker has become the most widely used estimation technique, and it is very closely related to the most frequently used size metric, namely Story Points (SP). Furthermore, SP plays a crucial role in gauging the size and complexity of user stories, which are central artifacts in defining user requirements in ASD [6].

This manuscript builds upon our initial research presented at the Iberoamerican Conference on Software Engineering (CIbSE) in 2023 [1], significantly expanding its scope to encompass crucial strategies for enhancing the accuracy of effort estimation in software projects. Our investigation uniquely addresses two primary goals: first, we carefully assess the current methodologies applied to software effort estimation within the agile development landscape in Colombia; second, we pinpoint and elaborate on the potential avenues for refining these estimation practices. To our knowledge, this study represents the first of its kind in the region, specifically focusing on Agile Effort Estimation (AEE). Of particular importance is our comparative analysis of the existing literature on effort estimation in Agile Software Development (ASD). Here, we identify the assortment of techniques employed, the challenges faced, and, critically, the prospects for advancement in this field.

In this study, we collected and scrutinized data spanning five critical domains pertinent to effort estimation in Agile Software Development (ASD): (1) the array of techniques employed for effort estimation; (2) the accuracy levels achieved in such estimations; (3) the predictors of effort utilized in these processes; (4) the datasets. i.e., comprehensive compilations of estimations from preceding projects instrumental in conducting effort estimation activities; and (5) the nuanced distinctions between our discoveries and those documented in analogous studies from various regions. Uniquely, our research method involved the deployment of an online survey as the principal data collection instrument. This approach enabled us not only to amass detailed insights into the aforementioned domains but also to capture the demographic profiles of individuals occupying diverse technical and managerial positions within the ASD sphere. It is paramount to underscore that our investigation stands as the first to exhaustively cover these five areas of interest. Furthermore, this study distinctively juxtaposes our findings with those of previous research, marking a pioneering effort in comparative analysis within this field.

Recognizing the pivotal role of estimation accuracy in the success of software projects, our analysis of the survey-based study has allowed us to identify crucial factors that hold the potential to enhance the accuracy of effort estimation significantly. We propose concentrating on four distinct yet interconnected areas for improvement. Firstly, we advocate for the standardization of measurement techniques within agile frameworks, ensuring a uniform approach to software estimation. Secondly, we emphasize the importance of utilizing comprehensive datasets, which serve as a foundation for refining estimation accuracy. Thirdly, we recommend the adoption of methodologies capable of quantifying estimation accuracy, thereby furnishing objective insights into the reliability of the estimates produced. Lastly, but equally important, is the imperative for robust knowledge management practices in the realm of effort estimation. By fostering an environment where information and expertise are systematically managed and shared, organizations can further elevate the accuracy and reliability of their estimation processes. This holistic approach not only aims at refining the estimation process itself but also at bolstering the overall quality and predictability of software project outcomes.

The paper is organized as follows. Section 2 presents the studies most related to software effort estimation in agile contexts. Section 3 describes the research methodology. Section 4 reports the most significant results of the study. Section 5 analyzes the main findings, elucidating their significance and implications. Section 6 pivots to exploring potential avenues for enhancing estimation accuracy, drawing directly from our study's findings. Section 7 addresses potential threats to the study's validity, ensuring a balanced and critical view of our work. Finally, Section 8 encapsulates our conclusions, summarizing the key insights and their broader impact on the field of agile software development and effort estimation.

2. Related work

There is extensive literature of effort estimation based on surveys. Most of these studies have been carried out on traditional software development. Molokken and Jorgensen reported studies of this type since 1984 [8]. As traditional software methods are still in used, there are recent publications with the same scope [9], [10]. On the contrary, the literature review indicates that there are few studies similar to ours regarding estimating effort in ASD.

The first of them is the research by Usman et al., which validates a previous study conducted by the same authors [11]. In this work of 2014, they surveyed 60 agile practitioners with experience in effort estimation from 16 different countries belonging to the five continents. Results showed that Planning Poker, Analogy, and Expert Judgment are frequently used estimation techniques in ASD, while SP is the most frequently used size metric. Moreover, the expertise level of the team and prior experience are the most commonly used cost drivers. More than half of the respondents believe that their effort estimates on average are under/overestimated by an error of 25% or more. Finally, they found that most agile teams take implementation and testing activities into account when estimating effort, and this is mostly done at the iteration and release planning levels in ASD.

The second study was carried out in 2018 by Vera et al. [12]. They conducted an empirical study that identified the most frequent approaches to effort estimation in Chilean companies. Their main findings reveal that companies use Expert Judgment mainly due to their inability to count on trustworthy historical data. This inability to record reliable information is usually a consequence of the

unplanned human resource sharing performed by the small software companies. It is worth noting that this study included all kinds of software projects.

In 2019, Prokopova et al. [13] conducted a study to analyze the project estimation process within Czech companies. In this research, 91 companies from six distinct areas in the information technology sector were surveyed. They found that the most widely used estimation technique is Expert Judgment with 57% incidence. Overall, 48% of companies report implementation time as a key factor for the estimation. The most common cause (39%) of delivery delays in software projects is the requirements volatility. In general, 54% companies failed to complete a quarter of projects in the estimated timeframe. It is important to note that this study considered both agile and traditional projects.

Regarding effort estimation accuracy levels in ASD, Ramessur et al. [14] analyzed 15 projects in a well-known Mauritanian company and uncovered eighteen factors that influence iterations effort estimation and promote better estimation accuracy levels. These factors mainly refer to stakeholders, team characteristics, code base features, requirements aspects, and managerial factors.

Finally, there are several studies about effort estimation on User Stories. Majchrzak et al. [15] propose factors influencing User Story estimation. Their results show three major factor groups: management, knowledge, and requirements issues. Additionally, there are post-release defect factors that are directly related to inaccurate estimates: demotivation, stress, missing requirements, and inappropriate tests. Abrahamsson et al. [16] propose an ASD method for predicting development effort based on well structured User Stories. Mahnič and Hovelja [17] carried out an empirical study using Planning Poker for estimating User Stories, whose main finding is the correlation between the expertise of estimators and the estimation accuracy. Haugen [18] shows that the introduction of Planning Poker improves the estimation performance of the team on Extreme Programming (XP) environments.

3. Research methodology

This is a survey-based study that follows the general guidelines proposed by Linaker et al. [19] who adapted the survey methodology to software engineering research. Thus, when designing this study, we performed principal steps that include (1) Defining its objective and research questions, (2) Determining the target population, (3) Developing clear and relevant survey questions aligned with the objective, (4) selecting the appropriate questionnaire type, (5) Determining the execution method, (6) Evaluating the questionnaire, and (7) Considering the questionnaire length and sequence of questions. The Linaker et al. includes guidelines for analyzing survey data, drawing conclusions, and documenting and reporting.

The entire design of this research is inspired by a survey study conducted in 2015 by Usman et al. [11], aimed at validating in a practical context the results of a previous SLR [20] that compiled and detailed the state of the art regarding estimation techniques, effort predictors, accuracy measures and agile methods used in ASD. Following a similar approach, we decided to validate and extend, asking Colombian agile practitioners, the results of an SLR performed by Fernandez-Diego et al. [6] that was published shortly before the beginning of this research. In addition to being a recent SLR, there is a couple of additional circumstances that make it suitable for our survey design. First, it is an update of the aforementioned SLR [20] that keeps the same research questions. Second, it covered primary studies published between December 2013 and January 2020. Furthermore, to allow direct comparisons between our study and the results of this SLR, we decided to include the same research questions.

3.1. Research questions

As we have said, the main motivation for our study is to know how Colombian ASD practitioners estimate effort. Given the lack of research in this regard, we find it both relevant and useful to conduct a survey study that, in the context of agile methods, inquiries about the similar aspects investigated by other studies at a worldwide level.

With the above in mind, we initially selected four research questions as formulated by Usman and Fernandez-Diego [20] [6]. We then adapted two of these original questions to focus specially on effort estimation. Moreover, to enhance our research's scope, we formulated a fifth question, assessing how our findings compare with similar studies globally. This led to a total of five comprehensive research questions guiding our study. These are formulated below:

- RQ1: What techniques have been used to estimate effort in ASD?
- RQ2: What is the accuracy level of effort estimation techniques in ASD?
- RQ3: What effort predictors have been used for effort estimation in ASD?
- RQ4: What are the characteristics of the datasets used for effort estimation in ASD?
- RQ5: Are there significant differences between the results of our study and previous studies conducted in other regions?

It should be noted that RQ3 aims to find out factors used to forecast the amount of work, time, or resources needed to complete a software endeavor accurately. Such predictors are useful for project managers and developers in effective planning, budgeting, and resource allocation. Examples of this kind of factors are team experience, development environment and the complexity level of the project to be carried out.

3.2. The survey and the respondents

An online survey was sent to 146 ASD practitioners from diverse technology areas and regions of the country who have worked on ASD projects performing, among others, software estimation activities.

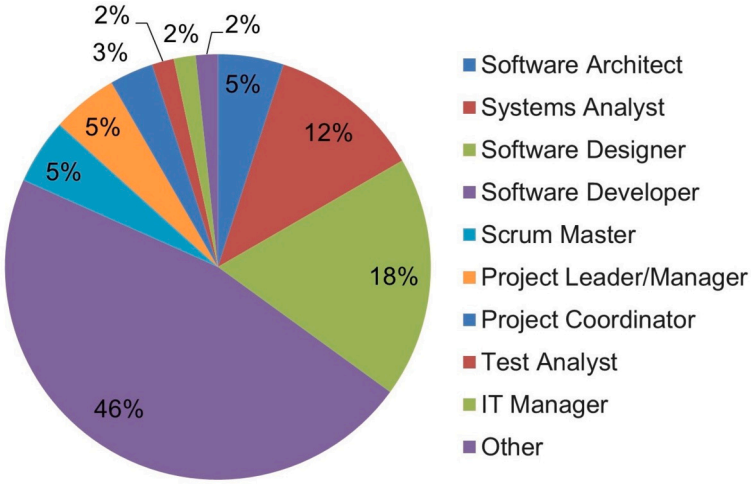

Fig. 1. Roles of the Respondents. (For interpretation of the colors in the figure(s), the reader is referred to the web version of this article.)

To identify the target population for our study, we compiled a list of individuals who had either academic or professional connections with us. Furthermore, we verified that each person was actively employed in roles related to software development, either within companies or government entities. This method of participant selection aligns with 'convenience sampling', a technique characterized by choosing individuals who are readily accessible and easy to reach [21].

The survey has 23 questions divided into five sections. The first section contains 7 questions related to the demographic profile, inquiries about the profession, educational level, age, gender, and software development experience of the respondents. The other four sections comprise 16 questions, and they are all related to software effort estimation and collect information to help answer the research questions. These sections inquire about techniques, tools, standards, and effort drivers, among other topics.

Respondents were given a pre-defined list of values for each question to indicate their answer, and could also include any additional values they felt were relevant. This applies to all questions related to demographics and software effort estimation.

The survey was available for three weeks and 60 complete responses were collected. These answers correspond to the 41% of the sample initially planned. The survey was distributed in Spanish as this is the mother tongue of the participants. For non-Spanish speaking readers of this paper, an English version of the entire questionnaire is available online. [22].

4. Results

This section presents the survey results organized by demographics, effort estimation data grouped by research question, and correlation analysis of the most significant variables identified in this study.

4.1. Demographics

Sixty Colombian ASD practitioners from different regions answered the survey. Geographically, almost all the respondents work in the Andean region of the country, where the most densely populated cities are located.

In respect of *age and education level*, 50% of the respondents are over 36 years old, 45% are between 25 and 35 years old, and 5% of them are under 25. Furthermore, more than 90% of them have at least a bachelor's degree and 63% have earned a post-graduate degree.

Regarding *roles*, the respondents are mainly developers (46.7%), software designers (18.3%), and business analysts (11.7%). There is minor representation of other roles, such as software architects (5%), scrum masters (5%), project leaders (5%), IT managers (2%), among others (Fig. 1).

Regarding the *business area* where they work, most of them are in the information technology (59%) and financial sectors (27%). Other minor areas reported were government and the medical and healthcare industries (Fig. 2).

Finally, concerning *experience in software engineering*, 81.7% of the respondents reported more than 5 years working in the software industry and regarding *effort estimation experience*, 48.3% of the respondents reported more than 5 years of experience, 26.7% have at least 2 years, and 13.3% less than one year doing estimation tasks (Fig. 3).

4.2. Effort estimation

This section presents the results of the survey grouped by research question.

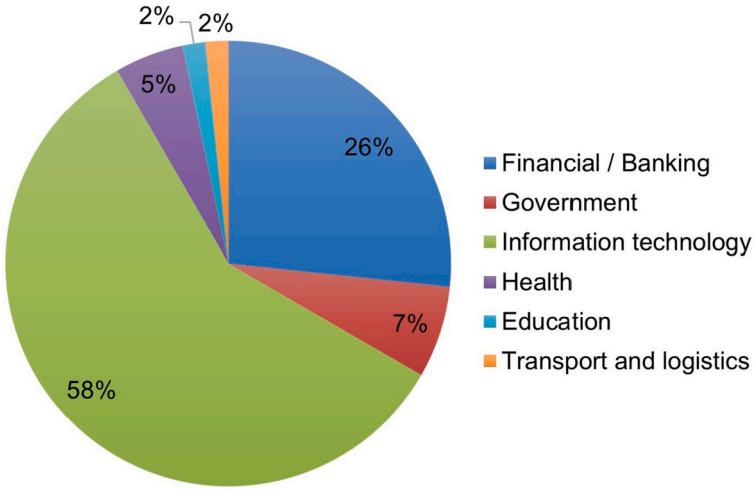

Fig. 2. Business Area of the Respondents.

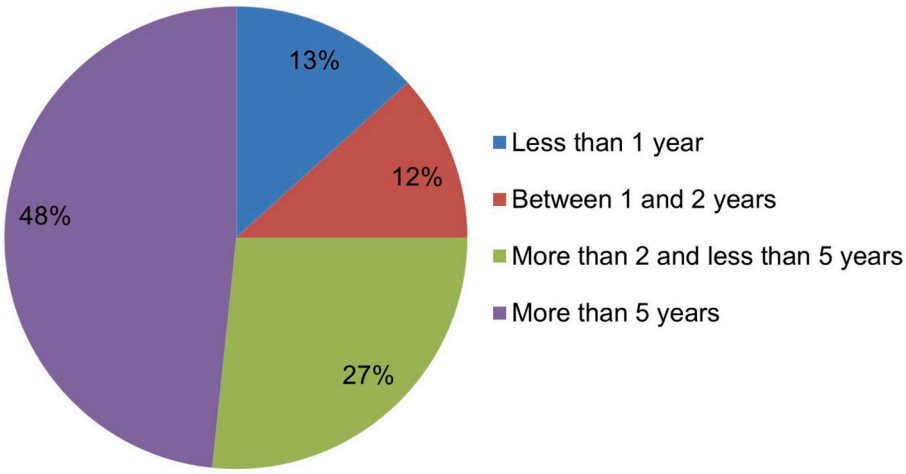

Fig. 3. Experience on SDEE of the Respondents.

4.2.1. RQ1: what techniques have been used to estimate effort in ASD?

This research question delves into the multifaceted nature of effort estimation within agile environments, aiming to uncover not merely the array of techniques utilized but also to examine closely related elements that influence the estimation process. These elements include the types of artifacts leveraged during estimation, the hierarchical levels at which estimates are formulated, the various units employed to quantify project sizes, and the specific agile methodologies underpinning these estimation practices. Thus, the survey includes six questions designed to address this particular research inquiry. First, we asked them about the effort estimation techniques used in their agile projects. The results show that the top 5 techniques used by the respondents are Expert Judgment - Delphi (66.7%), Planning Poker (53.3%), Analogy (35%), Top-Down (26.7%), and Regression (21.7%). Fig. 4 shows the results for all effort techniques.

Second, we inquired about the agile frameworks used by the respondents. The most used frameworks are Scrum (100%), Kanban (41.7%), Extreme Programming (36,6%), and Test-Driven Development (15%). Other agile methods are used by less than 10% of the respondents, including Lean Software Development, Crystal, Dynamic Systems Development Method (DSDM) and Feature Driven Development (FDD). Frameworks for scaling agile, such as SAFe, have been reported in only a few cases.

Third, we asked at what planning levels the respondents perform the effort estimation. The results indicate that 78.3% of the respondents estimate at the User Story level, 66.7% estimate at project level and 45% estimate at release and iteration level (Fig. 5).

Fourth, we inquired about the software lifecycle activities in which the respondents performed effort estimation. The most relevant software activities in which estimation is performed are development (88.3%), followed by analysis (60%), design (53.3%), testing (51.7%), documentation, maintenance (36.7%), and training (18.3%).

Fifth, we inquired about tools to support the effort estimation process. We found that spreadsheets are the most used (90%), followed by customized tools (20%). Specialized tools for estimation are used to a lesser extent (20% for open source tools and 10%

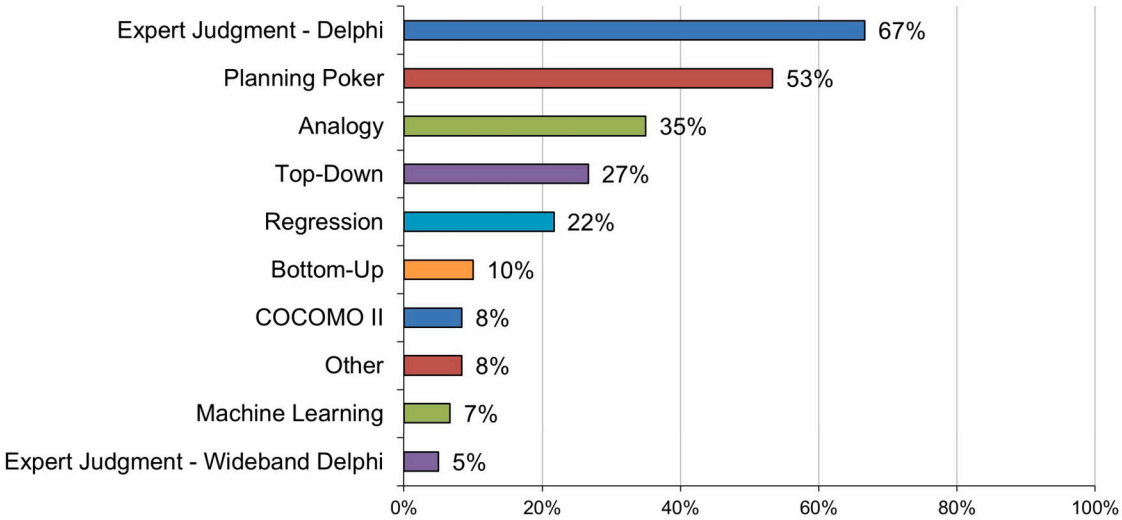

Fig. 4. Techniques Used for Effort Estimating.

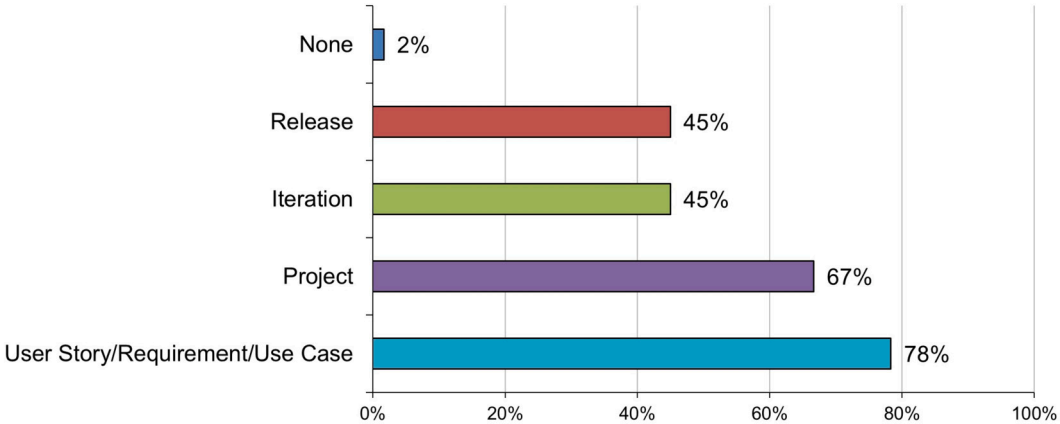

Fig. 5. Levels of Estimations.

for proprietary ones). 5% of the respondents use no tools and only 3.3% of them use project management or application lifecycle management tools such as Kanban, MS Project or Azure DevOps tools.

Finally, we asked about software measurement units used in agile projects. The most used unit is Story Points (58.3%), followed by Use Case Points (33.3%), Function Points (30%), and Lines of Code (21.7%). A significant percentage of respondents (16.7%) use no unit and some of them reported 'working hours' as an additional measurement unit, although this one is not defined as a software measurement unit. Our investigation does not delve into the specific manner in which each respondent utilizes these measures. It's plausible that identical measures are applied differently, influenced by factors such as the chosen estimation technique, the intended purposes, and even the participants' potential misunderstandings about the measure's nature and properties.

4.2.2. RQ2: what is the level of accuracy of effort estimation techniques in ASD?

To answer RQ2, the survey includes three general questions and one specific for the Planning Poker technique.

The results show that the accuracy level is high for 23.3% of the respondents, medium for 53.3%, and low or very low for 11.7% of them. Additionally, 11.7% of the respondents do not have a clear idea about the accuracy of their estimates (Fig. 6).

With respect to techniques for measuring the effort estimation accuracy, most of the respondents (70%) use no technique to calculate the estimation accuracy levels, while 18.3% use variance analysis, 8.3% Prediction at level X or PRED(X), and 8.3% Magnitude of Relative Error (MRE).

Regarding factors affecting accuracy of effort estimation, the results show that the most relevant factors are the definition of requirements (80%), followed by the size and experience of the project team (58.3%), user involvement (45%), excessive optimism or pessimism (36.7%), project management (35%), and formality of the estimation process (23.3%). Fig. 7 shows all factors that somehow influence effort accuracy level.

The data collected indicate that 56.6% of the respondents have used Planning Poker as an effort estimation technique. As in other studies, this result confirms that this technique is by far the most used among agile practitioners [11]. Our survey inquires about the

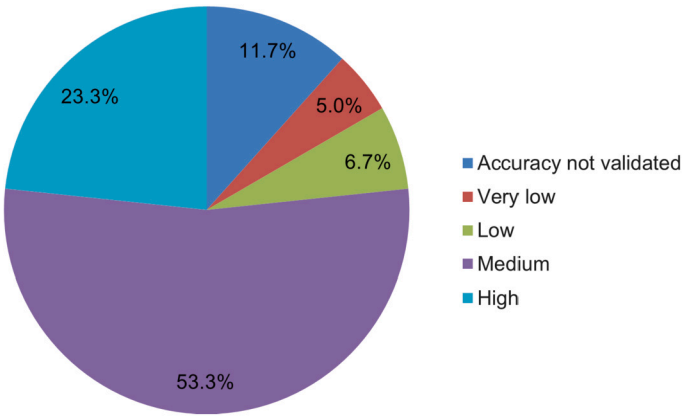

Fig. 6. Effort Accuracy Level.

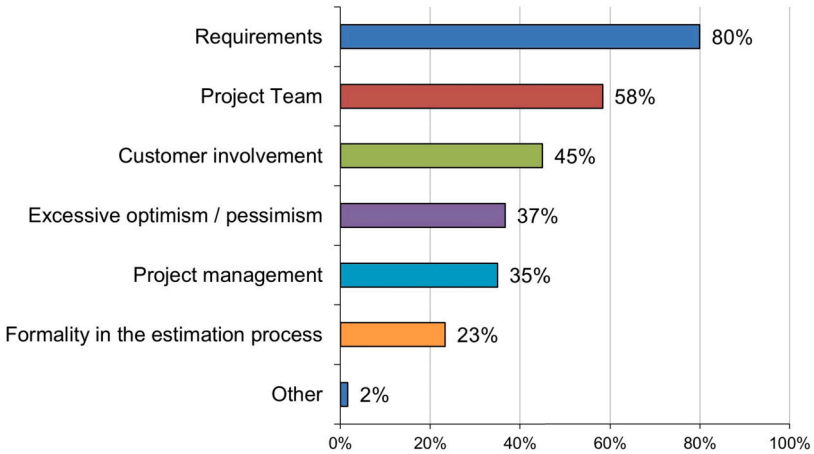

Fig. 7. Factors Influencing Effort Accuracy Level.

Table 1
Results of Accuracy Level for Planning Poker.

Estimation	Accuracy level					
	Very High	High	Medium	Low	Very low	N/A
Duration	12%	41%	**44**%	0%	3%	0%
Cost	6%	12%	**56**%	14%	3%	9%
Resources	3%	38%	**45**%	6%	0%	8%

perceived accuracy level of duration, cost, and resources estimates, when using planning poker. Table 1 shows the results distributed by type of estimation and accuracy, when the Planning Poker technique is used.

4.2.3. RQ3: what effort predictors have been used for effort estimation in ASD?

We formulate two questions to know which effort predictors are considered fundamental in ASD. Regarding the perception of the predictors' importance in the effort estimation activities, results show that the team experience is a fundamental factor for almost all of the respondents (95%). The second most selected factor is software complexity (83.3%). The third, fourth, and fifth places in the list are type of development (65%), team size (65%), and agile development method (53.3%), respectively (Fig. 8).

Concerning the predictors actually used by the respondents, the resulting list is almost the same. They consider team experience as the main factor (86.7%), followed by software complexity (70%), development type, i.e., new or maintenance (60%), and team size (55%). Other factors are considered but to a lesser extent, including development method (agile or traditional), software domain (business sector), and programming language, among others (Fig. 9). The four most important effort predictors are the most used in practice by the respondents. Two noteworthy differences refer, on the one hand, to the agile method used, and on the other, to the project duration. The agile approach adopted by the team seems to be less important in practice, while the project duration turns out to be more important in practice than in theory.

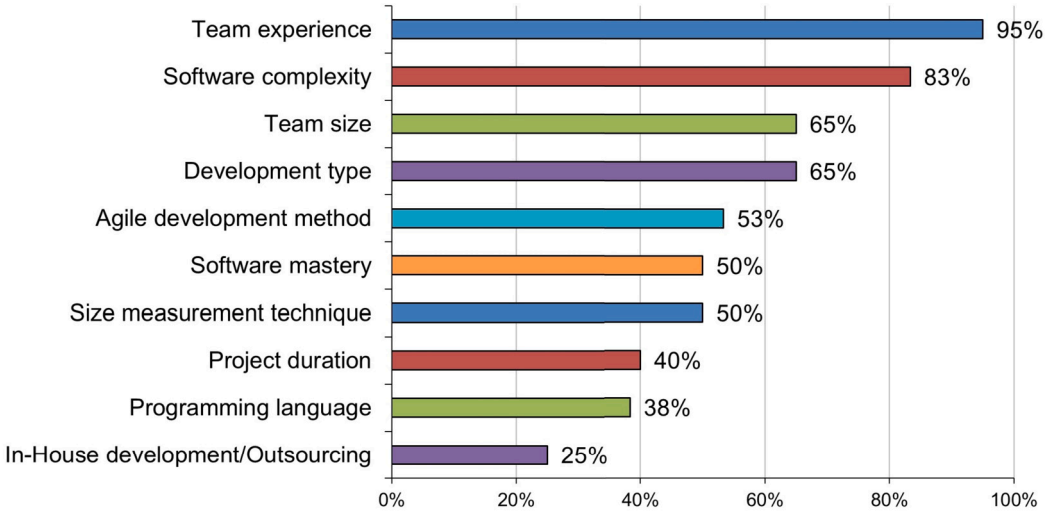

Fig. 8. Effort Predictors Considered Fundamental in ASD.

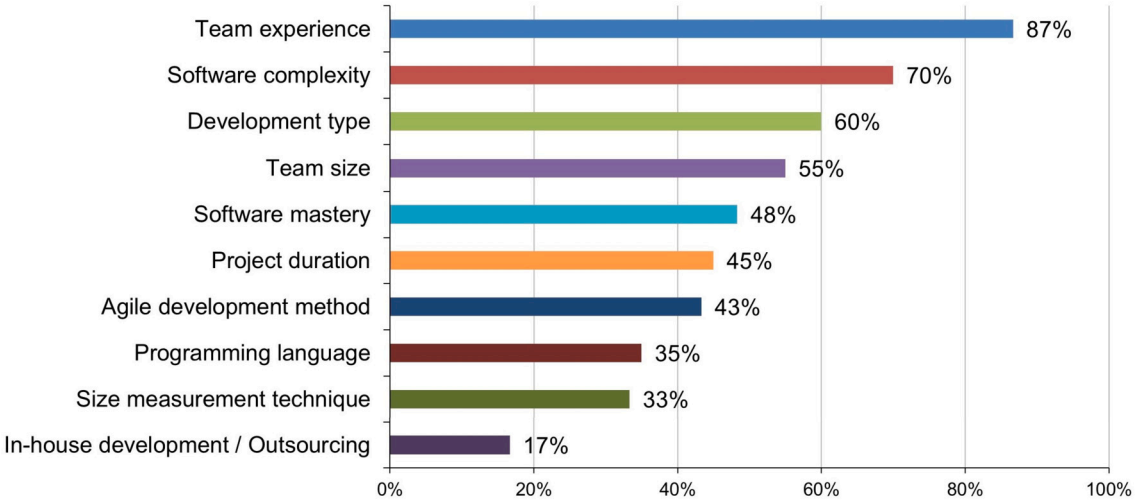

Fig. 9. Effort Predictors Used in ASD.

4.2.4. RQ4: what are the characteristics of the datasets used for effort estimation in ASD?

First of all, the most important finding is that the majority of the people surveyed (83.6%) have never made an effort estimate based on past project datasets. However, from the responses of people who have used datasets, we identified two characteristics of the datasets used. First, the organizations where the respondents work own these datasets. Second, the datasets are used to estimate by analogies. Furthermore, only 3.6% of the respondents have used licensed datasets, 7.2% free access datasets, and 10.9% datasets owned by the company.

Regarding how the datasets have been used, we found that 12.7% of the respondents use them to make comparisons among projects of the same company, 9.09% apply parametric effort estimation techniques by using the datasets, and 5.4% use the datasets to estimate effort based on comparisons with projects outside their companies.

4.2.5. RQ5: are there significant differences between the results of this study and previous studies conducted in other regions?

Estimation techniques. In this regard, Usman et al. [11] conclude that Planning Poker and Analogy are the most used techniques in ASD. Vera et al. [12] report Expert Judgment as the main technique for software effort estimation and, in some cases, Expert Judgment combined with Analogy. In our study, Expert Judgment and Planning Poker are the most used techniques for effort estimating. Our study and the one by Usman et al. [11] are entirely comparable, as the respondents chose from the same list of estimation techniques. The comparison with the study by Vera et al. [12] is partial because the list of techniques provided by their survey is unknown. In the study by Prokopova et al. [13], respondents had the option of choosing Expert Judgment and Analogy but not Planning Poker. Therefore, only the results obtained in relation to Expert Judgment are comparable in all studies, as shown in Fig. 10.

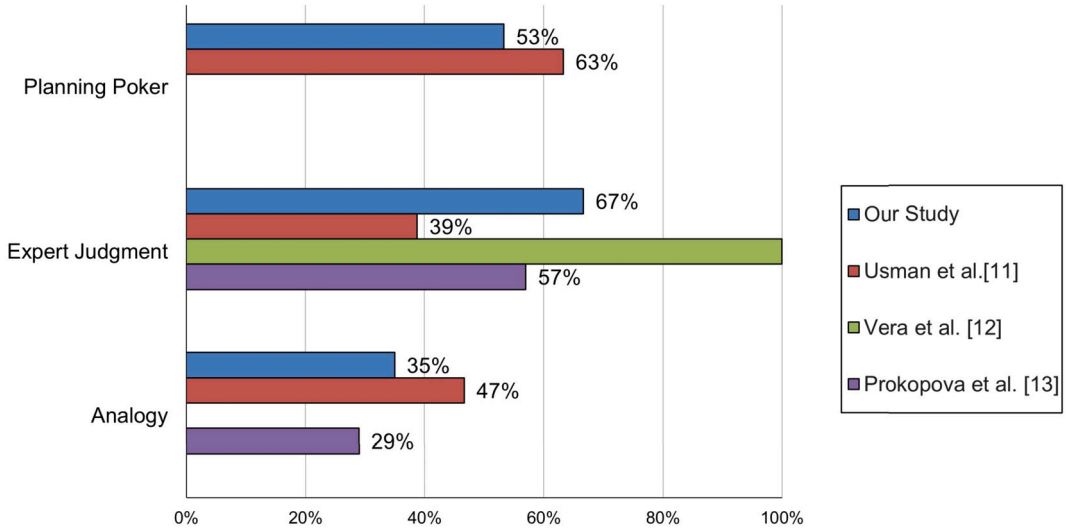

Fig. 10. Estimation Techniques in Related Studies.

Table 2
Correlations between Significant Variables.

Variables	p-value	Contingency Coefficient	Explanation
SWE Experience and SDEE Experience	$p<0.001$	0.653	Significant association (moderate positive correlation) between the SWE - Software Engineering Experience and Experience in Software Development Effort Estimation.
SWE Experience and Accuracy Level	$p=0.012$	0.497	Significant association (moderate positive correlation) between the Software Engineering Experience and the Perception about Estimation Accuracy Levels.
SDEE Experience and Accuracy Level	$p<0.001$	0.610	Significant association (moderate positive correlation) between the Software Development Effort Estimation Experience and the perception about Estimation Accuracy Levels.

Effort predictors. Our results point out that the most relevant is team experience, followed by software complexity, and development type (new or maintenance). Ramessur et al. [14] list similar factors affecting effort estimates and conclude that the prime factors that may affect the accuracy of the estimation are team experience, technical skills, and complexity of requirements.

Estimation Accuracy Level. Our results indicate that the perception of accuracy level of estimates is high for 23.3% of the respondents, medium for 53.3%, and low or very low for 11.7% of them. On the other hand, 11.7% of the respondents do not have a clear idea about the accuracy of their estimates. Usman et al. [11], as many other authors, report a tendency to underestimate effort. They also found that the combination of techniques and Story Points have a positive correlation with estimation accuracy. Vera et al. [12] indicate that almost half of companies overestimate the project effort, independently of the life cycle being used. This happens in some cases because the company uses a multiplier to overestimate projects to reduce the risk of the endeavor. Additionally, companies get no feedback from their estimations, and consequently they do not know exactly the estimations accuracy level.

Prokopova et al. [13] investigate the number of projects that are delivered on time. They found that more than half of companies (54%) do not complete 25% of their projects within their estimates, which is consistent with the trend identified in other related studies.

4.3. Correlations

A statistical correlation analysis was conducted to identify significant interdependencies between variables. The correlations were calculated using JASP [23], a free and open-source statistical analysis program supported by the University of Amsterdam. Possible relationships between specific variables were identified, considering that most of the variables are nominal categorical with more than two values. By using JASP, we defined contingency tables and this software program calculated the level of significance (p-values), contingency coefficient, and Chi-squared test to determine whether there is a statistical significance between the selected variables. Table 2 shows the results of the correlation analysis. Our results are similar to the Mahnič and Hovelja in terms of dependencies between respondents experience and effort estimation accuracy level [17].

In addition to the identified correlations, we analyzed the possible correlation between the estimation techniques used and the accuracy level of effort estimation. The analysis showed a p-value result of 0.833. This p-value indicates weak evidence against the null hypothesis, which states that there is no correlation between the variables. A p-value of 0.833 suggests that there is no statistically significant correlation between the variables being analyzed. The observed correlation between the variables is not

Table 3
Effort Estimation Techniques.

Topic		Findings of the Study
Type of Estimation Technique	Non-algorithmic	Frequently used. Expert Judgement (Planning Poker and Delphi) is preferred
	Algorithmic	Not frequently used
	Learning Oriented	They are rarely used
Estimation Accuracy	Level	The most representative perception: medium accuracy
	Techniques	Variance analysis and magnitude of the error
	Influencing aspects	Requirements and team size and experience

Table 4
Effort Estimation Predictors.

Factor	Most Relevant Aspects
Team	1. Experience (Technical skills, maturity, training, and knowledge)
Project	1. Complexity (Technical and functional)
	2. Type of project (new development, evolution)
	3. Domain (Industry)

statistically significant and any apparent correlation observed in the data may be due to random chance rather than a genuine relationship between the variables.

5. Discussion

This section offers an in-depth discussion framed by the research questions that steered the course of this study.

5.1. Effort estimation techniques

In ASD, non-algorithmic techniques are preferred since software project teams favor collaborative and consensus-based approaches to estimate the effort required to build software [6]. In this category, Expert Judgment-oriented techniques are most commonly used by ASD practitioners. Planning Poker and Delphi are the most used. Analogy and Top-Down techniques are at the second level of use.

In the realm of algorithmic techniques, academic literature emphasizes regressions and COCOMO as particularly significant. However, our survey reveals that their usage is less prevalent compared to non-algorithmic approaches. Interestingly, techniques based on function points are not commonly used.

Furthermore, learning-oriented techniques are not widespread enough in agile contexts. Instead of asking for specific techniques, our survey asks only for the use of learning-oriented techniques. Only 6.7% of the respondents have used them for software estimation.

An essential discovery from our research is that nearly all participants employ at least one estimation technique. This observation indicates that effort estimation is not only a common, but also a critical process in agile projects. Despite its recognized importance in the Colombian software industry, our study reveals that the estimation process is often not executed as effectively as it could be.

Regarding estimation accuracy, the perceptions of the respondents of high, average, and low levels of accuracy are 23.3%, 53.3%, and 11,7%, respectively. Moreover, 70% of the respondents use no technique for analyzing the accuracy of their estimates. Among the respondents who analyze accuracy, the most representative technique for measuring it is variance analysis (18.3%). In less than 17% of cases, the techniques used are oriented to measure the deviation or magnitude of the error. The aspects that most influence estimation accuracy are requirements (definition, specification, size, etc.), as well as team size and experience.

Table 3 shows the summary of the findings on the effort estimation techniques.

5.2. Effort estimation predictors

The factors that most influence the estimation process can be divided into two main groups: Those related to the team carrying out the software project and those that are connected to the characteristics of the project.

On the one hand, team experience includes aspects such as the technical skills of the members, their maturity as a team, as well as the quantity and quality of their training and the know-how of the team members about the software process to be used. This experience is the most important factor to consider in estimating effort, according to the results. On the other hand, the characteristics of the project have similar relevance. In this category, the complexity of the software to be built is the most significant factor. Another influential factor is whether the project is a new development, maintenance, or enhancement, mainly due to the level of uncertainty involved in each type of development. Lastly, in a third-place of relevance is the software domain, i.e., the targeted subject area of the software to be built.

Table 4 shows the summary of the findings on the effort estimation predictors.

Table 5
Characteristics of Related Studies.

Study	Instrument	Observations
Usman et al. [11]	Survey - Individual	Online questionnaire 60 people surveyed.
Vera et al. [12]	Interview-Company	Semi-structured interview, One person per company 10 people surveyed.
Prokopova et al. [13]	Survey-Company	Questionnaire sent to 91 companies.
Our study	Survey-Individual	Online questionnaire 60 people surveyed.

5.3. Effort estimation datasets and tools

We found that spreadsheets are the most popular tool for supporting the effort estimation process, possibly due to the ease and flexibility to build models with them. Spreadsheets are suitable for building simple applications to implement effort models, such as regression curves based on public or proprietary datasets. The use of proprietary tools is not common, perhaps due to the licensing prices of this type of tools.

With respect to past projects datasets, they are rarely used. This can be a strong reason to have low usage rates of parametric and learning-oriented effort techniques. In cases where datasets are used, proprietary datasets stand out, i.e., datasets based on information from previous projects of the company. Finally, datasets are employed mainly for productivity comparison purposes between past and current projects within the organization.

5.4. Planning level in ASD

As shown in Fig. 5, the most frequent planning level is User Stories / Requirements / Use Cases (78.3%). Likewise, there are high levels of planning on projects (66.7%), releases (45%), and iterations (45%). This might suggest that there is an interest in ASD for effort predictions at iteration, release, and project level. This in turn would confirm one of the main effort estimation goals in the early stages of the software life cycle: planning, as accurately as possible, the resources required by the software project. It would be interesting to know how planning is performed at the project level, as one of the main characteristics of ASD projects is uncertainty in scope. Lastly, it is worth noting that more than 98% of the people surveyed perform some type of planning. Only one respondent admitted that she makes no planning, at least at the levels considered in the survey.

5.5. Comparison with other studies

Table 5 shows the summary of related studies, in terms of the characteristics of the data collection processes.

Our results show that almost all surveyed people (88.3%) estimate coding activities, while analysis (60%), design (53.3%), and testing (51.6%) activities are taken into account for effort estimation in similar proportions. This is in line with Usman et al. [11], where coding activities are considered to be the most important for effort estimation (86.67%). However, they found that analysis (43,33%) and design (53,33%) activities are considered for effort estimation in a smaller proportion, while testing activities are the second most important factor.

With respect to factors that can affect effort estimation, they are similar in all cases. Likewise, the relevance assigned to these factors is also similar. Finally, team experience is the most important factor to consider in effort estimation according to our results and related studies [11], [14], [15].

As for estimation accuracy, we used a 5-point scale: very high, high, medium, low, and very low. For the same purpose, Usman et al. [11] use 2 major categories: overestimated or underestimated to express the accuracy of the estimation. Vera et al. [12] do not report how accurate the effort estimates are, instead, they concluded that the effort is always overestimated due to a multiplying factor that magnifies the estimates. In Prokopova et al. [13], only the percentage of projects not delivered on-time is reported.

6. Opportunities for improvement

From the results and findings of this study, it becomes evident that the most significant opportunities for improvement, particularly in the Colombian context, lie in assessing and enhancing the accuracy levels of software effort estimation. To improve effort estimation accuracy levels, existing literature suggests focusing on four key aspects: (1) standardizing the software measurement process for agile approaches, (2) using past projects datasets that allow improvements in estimation accuracy, (3) using techniques to measure estimation accuracy levels, and (4) knowledge management in effort estimation. The subsequent sections will delve deeper into these identified aspects, underlining how they serve as pivotal factors in improving the accuracy of effort estimation in software projects.

6.1. Standardized software measurement in ASD

Our study confirms that in ASD, the favored methods for assessing relative size are variations of the Expert Judgment. This includes techniques such as planning poker, a gamified approach based on consensus that uses the Fibonacci sequence to allocate point values to features or items [24]. A primary drawback of these techniques is their limited applicability in new measurement

Table 6
Example of Using Simple Function Points from a User Story.

Base Functional Components	Function Type	SFP
List upcoming classes	Elementary Process	4.6
See detailed information	Elementary Process	4.6
Trainer Profile	Logical File	7
Courses	Logical File	7
	Total Simple Function Points	23.2

scenarios. The estimates produced are relative, rendering them useful exclusively for the agile team that generated them, and challenging to adapt for other contexts. What this means is that creating a historical repository for future effort estimations is impractical, as comparing estimates made by different teams is not feasible.

Employing a standardized software size measurement technique, such as Function Points or COSMIC Points, enables the refinement of effort estimates. This can be achieved through algorithmic techniques like COCOMO, COCOMO II, SLIM, SEEM-SER, or Productivity Indexes. They allow organizations to make more informed decisions regarding the feasibility and execution of projects, drawing on historical data and specific project traits.

However, the adoption of an alternative size measurement technique in agile contexts should ideally possess key attributes, including user-friendliness, simplicity of the model, and effectiveness of results, among others. Simple Function Points (SFP), a size measurement technique for ASD, fulfills these criteria and has gained recognition in both industry and academia. It is endorsed by IFPUG, further attesting to its validity. [25]. In addition, it is worth to mention that recent studies have proposed new experimental size measures such as “Functional Stories” and “Issues” [26].

6.1.1. Simple function points

The SFP technique was designed by R. Meli in 2010 [27], [28], [29]. Subsequently, other authors carried out empiric studies that led to some refinements of the method including the first Simple Function Point Reference Manual [30]. In 2019 IFPUG acquired the SFP method and initiated a process of analysis and experimentation with it. In 2021, IFPUG released the official reference manual supported by this organization [31]. The SFP method adheres to the ISO14143-1 standard, which outlines the principles of Functional Size Measurement (FSM). This standard emphasizes measuring software size based on the user-required functions [32].

It is important to highlight that SFP is a technique that produces a functional size that has a high statistical convertibility with the traditional measure units such as COSMIC Points and IFPUG Function Points (FP) [33]. This means, for instance, that it is feasible to convert software size in function points to simple function points and vice versa. SFP estimates are useful very early in the software development process when details are not available to calculate the functional size. This technique is specifically designed to be agile, fast, lightweight, easy to use, and with minimal impact on software development processes. It is easy to learn, provides reliable, repeatable, and objective results [31]. Like IFPUG Function Points, it is independent of the technologies used and technical design principles. Likewise, SFP and story points can be used concurrently to maximize the effectiveness of each technique in optimizing effort estimates.

The SFP technique assumes that the functional value of a software product is directly proportional to the number of logical transactions (Elementary Process / Transactions) and logical files required. Therefore, identifying logical transactions and logical files from User Stories is a straightforward process. The SFP technique uses only two Base Functional Components (BFCs), namely Logical File and Elementary Process. A Logical File represents functionality provided to the user to meet internal and external data storage requirements. It is a user recognizable group of logically related data or control information maintained and/or referred within the boundary of the application being measured. Elementary Process is the smallest unit of activity that is meaningful to the user, constitutes a complete transaction, is self-contained, and leaves the business of the application being measured in a consistent state. As an example, Table 6 summarizes the analysis and calculations required to estimate the functional size of a user story, by using SFP. The user story, in its usual format, says *“As a trainer, I want my profile to list my upcoming classes and include a link to a detailed page about each so that prospective attendees can find my courses.”*

6.2. Effort estimation datasets

A dataset in the effort estimation context is well-defined repository containing mainly data about estimates and actual values of effort, costs, and duration of past projects. These sets usually contain qualitative and quantitative information regarding resources, artifacts, and techniques [34]. These datasets allow specialists to perform their accuracy analyses in a repeatable and comparable manner [35].

The software engineering community has widely adopted the use of repositories for estimating development effort, number of defects prediction, project maintenance costs, among others [36]. Several studies have been conducted regarding this type of datasets. Some of the most representative studies are focused on systematic literature reviews and mapping studies [37], [38], [39], industry and proprietary datasets [40] [41], [42], [43] and specific studies on datasets [44], [45], [46]. Historical data are valuable for improving the outcome and efficiency of effort estimation models. Nevertheless, our study’s findings suggest that in Colombia, there has been very limited use of effort estimation datasets. This clearly highlights an opportunity for improvement.

Table 7
Example of Accuracy Level Measurement.

User Story	Estimated Effort (Hours)	Actual Effort (Hours)	Effort MRE
US1	8	12	**33.33**
US2	7	6	**16.66**
US3	12	10	**20.00**
	MMRE (Effort)	**23.3%**	
	PRED(25)	**66.67%**	

There are two general approaches to take advantage of effort estimation datasets. The first is through industry datasets, public (Albrecht [47], Desharnais [48] China [49], COCOMO, [49] and NASA [49]) or proprietary (ISBSG [50] and QSM [51]). The second is through a historical repository of the organization itself that permanently records both estimates and actual values of its software efforts.

An effort estimation dataset serves two main purposes. Firstly, it enables teams to assess the accuracy of their estimates against the actual effort needed to implement User Stories, Use Cases, or functional and technical requirements. The insights gained from this analysis can then be applied to refine estimation techniques and their associated parameters. Secondly, such datasets facilitate benchmarking at various levels — be it project, release, iteration, or User Story — either against industry standards or within the organization's own projects. This benchmarking, which can utilize either industry-wide or proprietary datasets, is essential for conducting analyses of software development productivity.

6.3. Accuracy level measurement

Enhancing the estimation accuracy levels also requires having appropriate metrics that allow teams to analyze the differences between the estimates and actuals. These metrics are vital for teams to effectively analyze the disparities between their initial estimates and the actual outcomes. This kind of analysis is crucial, as it provides teams with concrete, quantitative evidence. With this data, teams can more accurately pinpoint the specific areas where their estimation methods need improvement. Moreover, this evidence-based approach ensures that any adjustments made are targeted and effective, leading to more reliable and accurate estimates in future projects. Such a methodical evaluation and adjustment process is essential for continuous improvement in estimation accuracy.

In terms of accuracy evaluation for estimation techniques, the de facto academic standards are The Mean Magnitude Reactive Error (MMRE), The Median Magnitude of Relative Error (MedMRE), and Prediction at level p (PRED(p)). However, according to our study, these standards are seldom applied in the Colombian software industry. In this regard, most authors support the usage of these effort estimation accuracy metrics [52], [53], [54] and, recently, they have been used to evaluate the accuracy of new estimation techniques based on learning-oriented approaches [55], [56]. Idri et al. investigate the properties and relationships among these metrics [57].

The key aspect behind these metrics is to calculate their values for each estimate-actual pair and determine if the results are within the organization's acceptable limits. These limits can be established based on the organization's historical data or software industry values. Table 7 shows an example of how the values of the MMRE and PRED(X) metrics can be calculated for three user stories, where the last column quantifies MRE.

An alternative to achieving continuous improvement in effort accuracy levels is to implement procedures to compare and select the most appropriate estimation technique for each scenario. This can be done using metrics similar to those suggested in this section.

6.4. Knowledge management in effort estimation

Based on the results of this study, the authors have identified the knowledge management in effort estimation as a relevant aspect to achieve an improvement in the use and application of techniques, tools, and datasets and thus carry out a more efficient and effective effort estimation process. Inaccurate estimates have been identified as a software projects failure factor [58], [59] and the knowledge management in effort estimation can be part of the solution to improve the literacy in this field.

This knowledge management can be materialized through different strategies. In any case, the selected strategy must implement a model that contains at least the following elements/functions related to effort estimation: (1) Training and education, (2) Analysis of tendencies and state of the art, (3) Management tools, (4) Standards and metrics definition, (5) Mentoring and team development, (6) Guidelines definition and (7) Auditing.

We suggest that this knowledge management be led by software project management offices (PMOs) and in some cases by areas specialized in software estimation (software measurement and estimation offices).

7. Threats to validity

Validity determines how much we can trust that the study findings accurately reflect the reality we want to describe or explain. For the particular case of a survey, many factors can affect that validity and they are discussed in this section, as well as, the actions we took to avoid or mitigate their effects.

External validity. First, the participants were mostly developers or people who play specific roles within software teams such as designers, analysts, architects, or scrum masters. Most of them have more than 5 years of experience in the software industry, with ages ranging from under 25 to over 45 years. Thus, the sample is quite heterogeneous in terms of experience, age, and job role. Second, the vast majority of them are in the Andean region where the largest population and the greatest industrial development of the country are concentrated. Despite the above, the results of our study cannot be fully applied to the entire population. Consequently, it is feasible that a replication of this study yields somewhat different results, in some of the analyzed aspects.

Internal validity. First, to reduce the threat that the subjects may not be competent enough to answer the questionnaire proposed, a list of potential respondents was built to ensure that participants had worked for software companies and had at least basic knowledge of effort estimation. Second, we collected specific demographic information regarding their career to be sure that the answers given by our respondents are reliable and suitable. Third, we conducted a pilot experiment with two graduate students and obtained feedback about the questionnaire itself and the time required to respond.

Construct validity. In this regard, we mitigated the associated threats by designing the questionnaire based on the results of an SLR. In addition, we guaranteed the relevance of each question included by associating it with one of the research questions formulated in this study. Moreover, the questionnaire items were iteratively improved so that we ensure the questions are valid, clear, unambiguous, and bias-free. Lastly, we write an introduction to the survey where we explained the purpose of the study, and clarified the concept of effort estimation.

8. Conclusions

We present a survey study aimed at determining how Colombian ASD practitioners carry out the effort estimation process within agile projects. The designed survey inquires about the effort estimation techniques used, the accuracy of the estimates, the factors that influence the accuracy of the estimates and whether datasets are used to support the estimation processes in any way. The survey was answered by 60 people who play various roles within Colombian software companies. We used convenience sampling and ensured that all participants work in the software industry.

With respect to estimation techniques, the ones based on Expert Judgment are the most frequently used. Within this category, Planning Poker stands out because it is used by more than half of the respondents and its effectiveness is considered medium-high when estimating duration, costs, and resources in ASD projects. Furthermore, the use of parametric techniques is representative (regression, COCOMO, Function Points Analysis). A minority of the respondents report using learning-oriented techniques and the use of proprietary techniques is marginal.

Regarding the agile processes, Scrum has been used by all participants, while Kanban and XP were reported by less than half of the respondents. The predominance of Scrum makes it the typical framework where the application of effort estimation techniques takes place. Moreover, effort estimation occurs primarily within the fine-grained planning process, that is, at the User Story or Use Case level. Data also suggests that estimation results are used, to a lesser extent, in planning iterations, releases, and the entire project. These results are consistent with studies that describe the way agile projects are planned and managed [11].

The respondents also reported that the activities for which effort estimates are carried out are development (90.9%), analysis phase (61.8%), design tasks (58.2%), and software testing work (56.4%).

Regarding software sizing, Story Points and Use Case Points are used to a great extent as units for estimating software size. Approximately, one-third of the surveyed people use Function Points, and LOC to a lesser extent. Additionally, about 20% of the respondents reported that they use no size measurement technique.

In terms of tooling, the use of specialized software to perform software estimation is not common. This activity is dominated by spreadsheets (90.9%) and, to a lesser extent, by software tools tailored to the needs of each organization, which can be based on spreadsheets.

Estimation accuracy is considered high for a fifth of the surveyed people, average for slightly more than half of them, and low or indeterminate for the rest of the respondents. However, about 71% of the respondents report that a formal accuracy analysis process is not carried out. The use of variability analysis was reported by 20% of them, while MRE and PRED(X) were reported by a small proportion of people. All factors that affect the accuracy of the estimates suggested in the survey are recognized as significant.

With respect to effort predictors, the most relevant in order of importance are team experience, software complexity, development type, team size, software domain, and software size measurement technique used.

With respect to datasets, we found that, in most cases, they are not used to support the effort estimation process. This indicates a low level use of parametric or learning-oriented techniques, which require historical data for calculating estimates or training the models. In the few cases where datasets are used, they are mostly public sources or historical data from the organization itself.

The general results suggest that although estimations are performed as an input for project planning, their accuracy is not formally evaluated. In this regard, the majority of respondents simply perceive that their estimates are acceptable or good, but there is no evidence of any effective process that supports such perceptions. We believe that the support of tools that facilitate data capture, as well as its subsequent analysis, could substantially improve the quality of the estimates.

The analysis of the survey results allowed us to identify three strategies that the Colombian software industry can implement to improve the estimation processes, in terms of accuracy levels. Firstly, the use of a standard method to measure the size of the software that complements the traditional ones used in ASD. This approach enables the use of alternative effort estimation methods other than those based on Expert Judgement. Secondly, the use of effort estimation datasets (either proprietary or public), since this allows the ASD teams to employ historical data for productivity analysis or complement the estimates through comparison and

analogy techniques at the project, release, or user story level. Lastly, the use of techniques for measuring the estimation accuracy levels is highly recommended as this allows agile teams to determine the effectiveness of the estimation method being used.

CRediT authorship contribution statement

Juan Cubillos: Writing – review & editing, Writing – original draft, Validation, Software, Resources, Project administration, Methodology, Investigation, Formal analysis, Conceptualization. **Jairo Aponte:** Writing – review & editing, Writing – original draft, Supervision, Project administration, Methodology, Investigation, Formal analysis, Conceptualization. **Diana Gomez:** Writing – original draft, Investigation. **Edwar Rojas:** Writing – original draft, Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

We want to express our immense gratitude to all the people who kindly responded to our request, and thus made this study possible.

References

[1] D. Gómez, J. Cubillos, J. Aponte, E. Rojas, Effort estimation in agile software development: the state of the practice in Colombia, in: CIBSE 2023, 2023, https://sol.sbc.org.br/index.php/cibse/article/view/24689.
[2] M. Chemuturi, Software Estimation Best Practices Tools & Techniques, J. Ross Publishing, Broward-Florida, 2009.
[3] J. Wen, S. Li, Z. Lin, Y. Hu, C. Huang, Systematic literature review of machine learning based software development effort estimation models, Inf. Softw. Technol. 54 (1) (2012) 41–59, https://doi.org/10.1016/j.infsof.2011.09.002.
[4] A. Trendowicz, Software Project Effort Estimation, Springer-Verlag, Berlin, Germany, 2014.
[5] O. Malgonde, K. Chari, An ensemble-based model for predicting agile software development effort, Empir. Softw. Eng. 24 (2) (2019) 1017–1055, https://doi.org/10.1007/s10664-018-9647-0.
[6] M. Fernandez-Diego, E.R. Mendez, F. Gonzalez-Ladron-De-Guevara, S. Abrahao, E. Insfran, An update on effort estimation in agile software development: a systematic literature review, IEEE Access 8 (2020) 166768–166800, https://doi.org/10.1109/ACCESS.2020.3021664.
[7] PMI, A Guide to the Project Management Body of Knowledge, 7th ed., Project Management Institute, Newtown Square, Pennsylvania, USA, 2021.
[8] K. Moløkken, M. Jørgensen, A review of software surveys on software effort estimation, in: Proceedings - 2003 International Symposium on Empirical Software Engineering, ISESE 2003, 2003, pp. 223–230.
[9] T. Vera, S.F. Ochoa, D. Perovich, Survey of Software Development Effort Estimation Taxonomies, March 2018.
[10] A. Saeed, W.H. Butt, F. Kazmi, M. Arif, Survey of software development effort estimation techniques, in: ACM International Conference Proceeding Series, 2018, pp. 82–86.
[11] M. Usman, E. Mendes, J. Börstler, Effort estimation in agile software development: a survey on the state of the practice, in: ACM International Conference Proceeding Series, vol. 27-29-Apri, 2015, p. 10.
[12] T. Vera, S.F. Ochoa, D. Perovich, Understanding the Software Development Effort Estimation in Chilean Small, 2018.
[13] Z. Prokopova, P. Silhavy, R. Silhavy, Analysis of the Software Project Estimation Process: A Case Study, vol. 984, Springer International Publishing, 2019.
[14] M.A. Ramessur, S.D. Nagowah, Factors affecting sprint effort estimation, Adv. Intell. Syst. Comput. 1089 (January 2020) 507–518, https://doi.org/10.1007/978-981-15-1483-8_43.
[15] L. Madeyski, M. Majchrzak, Factors influencing user story estimations: an industrial interview and a conceptual model, Cent. East. Eur. J. Manag. Econ. 4 (4) (2016) 261–280.
[16] P. Abrahamsson, I. Fronza, R. Moser, J. Vlasenko, W. Pedrycz, Predicting development effort from user stories, in: Int. Symp. Empirical Software Engineering. Meas., 2011, pp. 400–403, https://doi.ieeecomputersociety.org/10.1109/ESEM.2011.58.
[17] V. Mahnič, T. Hovelja, On using planning poker for estimating user stories, J. Syst. Softw. 85 (9) (2012) 2086–2095, https://doi.org/10.1016/j.jss.2012.04.005.
[18] N.C. Haugen, An empirical study of using planning poker for user story estimation, in: Proc. - Agil. Conf. 2006, vol. 2006, 2006, pp. 23–31.
[19] J. Linaker, S.M. Sulaman, R. Maiani de Mello, M. Host, Guidelines for Conducting Surveys in Software Engineering, Department of Computer Science, Lund University, 2015.
[20] M. Usman, E. Mendes, F. Weidt, R. Britto, Effort estimation in agile software development: a systematic literature review, in: ACM International Conference Proceeding Series, 2014, pp. 82–91.
[21] M. Saunders, P. Lewis, A. Thornhill, Research Methods for Business Students, 8th ed., Pearson Education, 2019.
[22] Survey - English version, [Online]. Available: http://it-jmca.com/UN/Paper2/Survey.pdf, 2022.
[23] JASP Team, JASP (version 0.16.4) [Computer software]. Retrieved from: https://jasp-stats.org/, 2022.
[24] P.M.I. Practice, Standard for Project Estimating, 1st ed., Project Management Institute, Newtown Square, Pennsylvania, USA, 2019.
[25] IFPUG, IFPUG [Online]. Available: https://ifpug.org/, 2023.
[26] W. Rosa, S. Jardine, Data-driven agile software cost estimation models for DHS and DoD, J. Syst. Softw. 203 (2023) 111739.
[27] R. Meli, Simple function points, in: UKSMA/COSMIC Conference - 22nd Annual Conference on Metrics and Estimating, 2011.
[28] R. Meli, Simple function point, in: Software Measurement European Forum, 2011.
[29] R. Meli, Simple function point: a new functional size measurement method fully compliant with IFPUG 4. x, in: Softw. Meas. Eur. Forum, No. January 2011, 2011.
[30] R. Meli, Simple Function Point-Functional Size Measurement Method Reference Manual-v01.01, 2014, p. 30.
[31] IFPUG, Simple Function Point (SFP) Counting Practices Manual, 2021.
[32] ISO/IEC, ISO14143-1 Information Technology — Software Measurement — Functional Size Measurement, 2007.
[33] A.Z. Abualkishik, et al., A study on the statistical convertibility of IFPUG function point, COSMIC function point and simple function point, Inf. Softw. Technol. 86 (2017) 1–19.

[34] L. Lavazza, L. Santillo, Historical data repositories in software engineering: status and possible improvements, in: Proc. 2012 Jt. Conf. 22nd Int. Work. Softw. Meas. 2012 7th Int. Conf. Softw. Process Prod. Meas. IWSM-MENSURA 2012, 2012, pp. 221–225.
[35] T. Menzies, M. Shepperd, Special issue on repeatable results in software engineering prediction, Empir. Softw. Eng. 17 (1–2) (2012) 1–17.
[36] A.K. Bardsiri, S.M. Hashemi, M. Razzazi, Statistical analysis of the most popular software service effort estimation datasets, J. Telecommun. Electron. Comput. Eng. 7 (1) (2015) 87–96.
[37] F. Gonzalez-Ladron-De-Guevara, M. Fernández-Diego, ISBSG variables most frequently used for software effort estimation: a mapping review, in: International Symposium on Empirical Software Engineering and Measurement, 2014.
[38] A. Idri, I. Abnane, A. Abran, Dealing with missing values in software project datasets: a systematic mapping study, Stud. Comput. Intell. 653 (2016) 1–16.
[39] N. Halimawan, F.W.P. Dharma, Wincent, N. Surantha, Model and dataset trend in software project effort estimation – a systematic literature review, ICIC Express Lett. 15 (10) (2021) 1109–1117.
[40] C. Quesada-López, M. Jenkins, Function point structure and applicability validation using the ISBSG dataset, in: Proceedings of the 8th ACM/IEEE International Symposium on Empirical Software Engineering and Measurement - ESEM '14, 2014.
[41] A. Abran, Data Collection and Industry Standards: the ISBSG Repository, Softw. Proj. Estim., 2015, pp. 161–184.
[42] M.K. Kodmelwar, S.D. Joshi, V. Khanna, Using cocomo dataset effort estimation for developing software, Int. J. Recent Technol. Eng. 7 (5) (2019) 111–113.
[43] H. Unlu, et al., Software effort estimation using ISBSG dataset: multiple case studies, in: 2021 Turkish National Software Engineering Symposium, UYMS 2021 - Proceedings, 2021.
[44] M.F. Bosu, S.G. Macdonell, Experience: quality benchmarking of datasets used in software effort estimation, J. Data Inf. Qual. 11 (4) (2019).
[45] A. Setiadi, W.F. Hidayat, A. Sinnun, A. Setiawan, M. Faisal, D.P. Alamsyah, Analyze the datasets of software effort estimation with particle swarm optimization, in: Proceedings - 2021 International Seminar on Intelligent Technology and Its Application: Intelligent Systems for the New Normal Era, ISITIA 2021, 2021, pp. 197–201.
[46] H.T. Hoc, V. Van Hai, H.L.T.K. Nhung, R. Jasek, Improving the Performance of Effort Estimation in Terms of Function Point Analysis by Balancing Datasets, LNNS, vol. 596, 2023.
[47] V. Khatibi Bardsiri, D.N.A. Jawawi, S.Z.M. Hashim, E. Khatibi, A PSO-based model to increase the accuracy of software development effort estimation, Softw. Qual. J. 21 (3) (2013) 501–526.
[48] K. Dejaeger, W. Verbeke, D. Martens, B. Baesens, Data mining techniques for software effort estimation: a comparative study, IEEE Trans. Softw. Eng. 38 (2) (2012) 375–397.
[49] E. Kocaguneli, T. Menzies, J.W. Keung, On the value of ensemble effort estimation, IEEE Trans. Softw. Eng. 38 (6) (2012) 1403–1416.
[50] ISBSG, ISBSG - international software benchmarking standards group, [Online]. Available: https://www.isbsg.org/, 2023. (Accessed 6 August 2023).
[51] QSM, QSM software project database — QSM SLIM-estimate, [Online]. Available: https://www.qsm.com/resources/qsm-database, 2023. (Accessed 6 August 2023).
[52] B.A. Kitchenham, L.M. Pickard, S.G. MacDonell, M.J. Shepperd, What accuracy statistics really measure, IEEE Proc. Softw. 148 (3) (2001) 81–85.
[53] E. Stensrud, T. Foss, B. Kitchenham, I. Myrtveit, An empirical validation of the relationship between the magnitude of relative error and project size, in: Proceedings - International Software Metrics Symposium, vol. 2002-January, 2002, pp. 3–12.
[54] T. Arnuphaptrairong, A Literature Survey on the Accuracy of Software Effort Estimation Models, Lecture Notes in Engineering and Computer Science, vol. 2, 2016, pp. 668–673.
[55] Y. Mahmood, N. Kama, A. Azmi, A.S.A.S. Khan, M. Ali, Software effort estimation accuracy prediction of machine learning techniques: a systematic performance evaluation, Softw. Pract. Exp. 52 (1) (2022) 39–65.
[56] P. Sampath Kumar, R. Venkatesan, Improving Accuracy of Software Estimation Using Stacking Ensemble Method, 2021, pp. 219–227.
[57] A. Idri, I. Abnane, A. Abran, Evaluating Pred(p) and standardized accuracy criteria in software development effort estimation, J. Softw. Evol. Process 30 (4) (2018) 1–15.
[58] J. Verner, J. Sampson, N. Cerpa, What factors lead to software project failure?, in: Proc. 2nd Int. Conf. Res. Challenges Inf. Sci. RCIS 2008, 2008, pp. 71–79.
[59] Standish Group, The Chaos Report, Standish Group Report, 2018.